

A1 Team 3

Project: Uncovering Latent Risk and Structural Regimes in Drug Regulation

Members: Abhi Jindal, Daaksha B. Arun, Mahesh Wadhokar, Tanya Chhabra

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

1. Data Loading

Loading EMA medicines dataset and identifying key variables.

```
import pandas as pd

file_path =
"/content/drive/MyDrive/Medicines_output_european_public_assessment_re
ports.xlsx"
df = pd.read_excel(file_path, header=8)
df = df.loc[:, ~df.columns.astype(str).str.contains("^Unnamed")]

print("Rows:", len(df))
print("Columns:", df.shape[1])
df.head()

Rows: 1988
Columns: 30

{"type": "dataframe", "variable_name": "df"}

print(df.columns)

Index(['Category', 'Medicine name', 'Therapeutic area',
      'International non-proprietary name (INN) / common name',
      'Active substance', 'Product number', 'Patient safety',
      'Authorisation status', 'ATC code', 'Additional monitoring',
      'Generic',
      'Biosimilar', 'Conditional approval', 'Exceptional
circumstances',
      'Accelerated assessment', 'Orphan medicine',
      'Marketing authorisation date',
      'Date of refusal of marketing authorisation',
      'Marketing authorisation holder/company name',
      'Human pharmacotherapeutic group', 'Vet pharmacotherapeutic
```

```
group',
    'Date of opinion', 'Decision date', 'Revision number',
    'Condition / indication', 'Species', 'ATCvet code', 'First
published',
    'Revision date', 'URL'],
    dtype='object')
```

2. EDA & Preprocessing Updates (M2 → M4)

M2 baseline:

- Text normalization (lowercasing, punctuation removal, stopwords filtering)
- TF-IDF vectorization
- Cosine similarity computation
- Observed instability and lack of global clustering by authorization status

M4 refinement:

- Dataset and regulatory labels remain unchanged
- Introduced embedding-based representation
- Applied both PCA (linear) and UMAP (non-linear) dimensionality reduction

```
TEXT_COL = "Condition / indication"
```

```
print("Total rows:", len(df))
print("Missing indications:", df[TEXT_COL].isna().sum())
print("Unique indications:", df[TEXT_COL].nunique())
```

```
df["text_length_words"] =
df[TEXT_COL].fillna("").str.split().str.len()
print(df["text_length_words"].describe())
```

```
Total rows: 1988
Missing indications: 12
Unique indications: 1886
count      1988.000000
mean         70.129779
std          93.608530
min           0.000000
25%          23.000000
50%          42.000000
75%          80.000000
max         1092.000000
Name: text_length_words, dtype: float64
```

```
import re
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS

STOPWORDS = set(ENGLISH_STOP_WORDS)
```

```

def clean_text(text):
    text = "" if pd.isna(text) else str(text)
    text = text.lower()
    text = re.sub(r"^[^a-z\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    tokens = [t for t in text.split() if t not in STOPWORDS and len(t)
> 2]
    return " ".join(tokens)

df["clean_text"] = df[TEXT_COL].apply(clean_text)

df[["Condition / indication", "clean_text"]].head()

{"summary": "{\n  \"name\": \"df[[\\\"Condition / indication\\\"],\n  \\\"clean_text\\\"]\", \n  \"rows\": 5, \n  \"fields\": [\n    {\n      \"column\": \"Condition / indication\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 5, \n        \"samples\": [\n          \"Treatment of adult and paediatric patients\nwith confirmed diagnosis of hereditary tyrosinemia type 1 (HT-1) in\ncombination with dietary restriction of tyrosine and\nphenylalanine.\", \n          \"Plaque psoriasisCosentyx is indicated\nfor the treatment of moderate to severe plaque psoriasis in adults who\nare candidates for systemic therapy.Psoriatic arthritisCosentyx, alone\nor in combination with methotrexate (MTX), is indicated for the\ntreatment of active psoriatic arthritis in adult patients when the\nresponse to previous disease modifying anti rheumatic drug (DMARD)\ntherapy has been inadequate.Axial spondyloarthritis (axSpA)Ankylosing\nspondylitis (AS, radiographic axial spondyloarthritis)Cosentyx is\nindicated for the treatment of active ankylosing spondylitis in adults\nwho have responded inadequately to conventional therapy.Non-\nradiographic axial spondyloarthritis (nr-axSpA)Cosentyx is indicated\nfor the treatment of active non-radiographic axial spondyloarthritis\nwith objective signs of inflammation as indicated by elevated C-\nreactive protein (CRP) and/or magnetic resonance imaging (MRI)\nevidence in adults who have responded inadequately to non-steroidal\nanti-inflammatory drugs (NSAIDs).\", \n          \"Ebvallo is indicated\nas monotherapy for treatment of adult and paediatric patients 2 years\nof age and older with relapsed or refractory Epstein-Barr virus\npositive post-transplant lymphoproliferative disease (EBV+ PTLD) who\nhave received at least one prior therapy. For solid organ transplant\npatients, prior therapy includes chemotherapy unless chemotherapy is\ninappropriate.\", \n          \"semantic_type\": \"\", \n          \"description\": \"\", \n          \"column\":\n            \"clean_text\", \n          \"properties\": {\n            \"dtype\":\n              \"string\", \n            \"num_unique_values\": 5, \n            \"samples\":\n              [\n                \"treatment adult paediatric patients confirmed diagnosis\nhereditary tyrosinemia type combination dietary restriction tyrosine\nphenylalanine\", \n                \"plaque psoriasiscosentyx indicated\ntreatment moderate severe plaque psoriasis adults candidates systemic\ntherapy psoriatic arthritiscosentyx combination methotrexate mtx

```

```

indicated treatment active psoriatic arthritis adult patients response
previous disease modifying anti rheumatic drug dmard therapy
inadequate axial spondyloarthritis axspa ankylosing spondylitis
radiographic axial spondyloarthritis cosentyx indicated treatment
active ankylosing spondylitis adults responded inadequately
conventional therapy non radiographic axial spondyloarthritis axspa
cosentyx indicated treatment active non radiographic axial
spondyloarthritis objective signs inflammation indicated elevated
reactive protein crp magnetic resonance imaging mri evidence adults
responded inadequately non steroidal anti inflammatory drugs
nsaids\",\\n          \\\"ebvallo indicated monotherapy treatment adult
paediatric patients years age older relapsed refractory epstein barr
virus positive post transplant lymphoproliferative disease ebv ptld
received prior therapy solid organ transplant patients prior therapy
includes chemotherapy unless chemotherapy inappropriate\\\"\\n          ],\\
n          \\\"semantic_type\\\": \\\"\\\",\\n          \\\"description\\\": \\\"\\\"\\n
}\\n      }\\n    }\\n}\\\", \"type\": \"dataframe\"}

```

3. Analysis & Experiments

This section evaluates whether therapeutic similarity derived from indication text forms patterns that align with regulatory outcomes.

3.1 TF-IDF Baseline (Cosine Similarity)

TF-IDF provides a high-dimensional frequency-based representation of indication text. Cosine similarity is used to evaluate lexical similarity between medicines.

This baseline tests whether surface-level word similarity aligns with authorization outcomes.

```

from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(
    min_df=2,
    max_df=0.95,
    ngram_range=(1,2)
)

X_tfidf = tfidf.fit_transform(df[\"clean_text\"])

print(\"TF-IDF shape:\", X_tfidf.shape)

TF-IDF shape: (1988, 11872)

from sklearn.metrics.pairwise import cosine_similarity
import numpy as np
import pandas as pd

sample_size = min(500, len(df))
sample_idx = np.random.choice(len(df), sample_size, replace=False)

```

```

S_tfidf = cosine_similarity(X_tfidf[sample_idx])
sim_values = S_tfidf[np.triu_indices_from(S_tfidf, k=1)]
print(pd.Series(sim_values).describe())
count      124750.000000
mean         0.019652
std          0.047888
min          0.000000
25%          0.002007
50%          0.008573
75%          0.021107
max          1.000000
dtype: float64

```

3.2 Embedding-Based Representation

Embedding representations capture contextual semantic relationships beyond token frequency.

This refinement evaluates whether deeper semantic encoding improves structural coherence and regulatory differentiation.

```

from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer("all-MiniLM-L6-v2")

X_emb = model.encode(
    df[TEXT_COL].fillna("").astype(str).tolist(),
    show_progress_bar=True
)

X_emb = np.array(X_emb)

print("Embedding shape:", X_emb.shape)

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your
settings tab (https://huggingface.co/settings/tokens), set it as
secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
access public models or datasets.
  warnings.warn(

```

```
{"model_id":"edf25001fb05440fa86ddc89e0bf468d","version_major":2,"version_minor":0}
```

```
{"model_id":"b59b26af15454eb086cfef8d8846658b","version_major":2,"version_minor":0}
```

```
{"model_id":"80ab06b476d0478483ab42de37112832","version_major":2,"version_minor":0}
```

```
{"model_id":"f1d915d5076b4acbb7fcc9d32bec18a1","version_major":2,"version_minor":0}
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

```
{"model_id":"d1166467f2f840e9a86c7e29b0cd3159","version_major":2,"version_minor":0}
```

```
{"model_id":"6226557dd9c24c9e96dc47465a1a5f61","version_major":2,"version_minor":0}
```

```
{"model_id":"4a8cd24f85ff48099515620b2d2184e6","version_major":2,"version_minor":0}
```

BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2

Key	Status	
-----+-----+-----		
embeddings.position_ids	UNEXPECTED	

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.

```
{"model_id":"92ce3a0dc2d14c2a9d340cf26fef553f","version_major":2,"version_minor":0}
```

```
{"model_id":"ae4a7c88c5d84e2cb4be7a04cf271570","version_major":2,"version_minor":0}
```

```
{"model_id":"7d013e9d064d4ce9b3969a8ad72f4d8a","version_major":2,"version_minor":0}
```

```
{"model_id":"436ca09456f5468ab27758115f47e38e","version_major":2,"version_minor":0}
```

```
{"model_id":"7d5ceab90b454acbab2269d97c758e69","version_major":2,"version_minor":0}
```

```
{"model_id": "ca45a7bd1a5c46408fe86eb2f5624370", "version_major": 2, "version_minor": 0}
```

Embedding shape: (1988, 384)

3.3 PCA (Linear Structure)

Principal Component Analysis (PCA) is applied to scaled embedding features to inspect dominant linear semantic directions.

The first two components explain approximately 9.5% of total variance, indicating that semantic information is distributed across many dimensions.

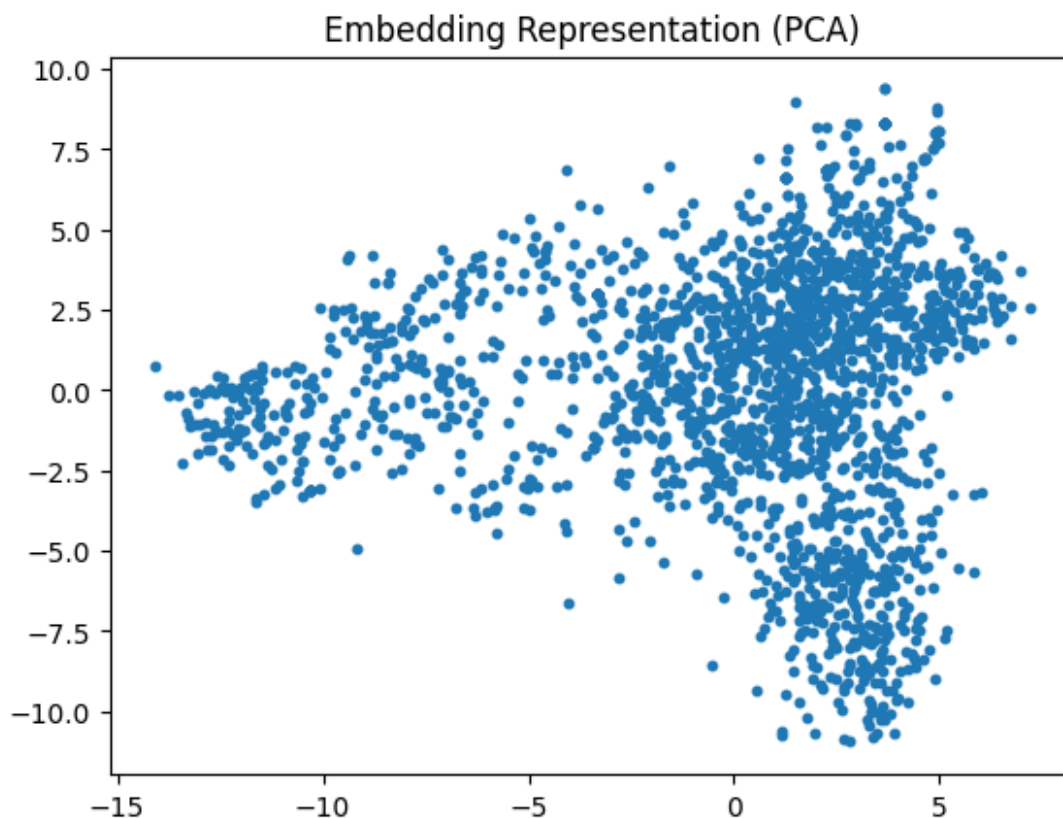
```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

X_emb_scaled = StandardScaler().fit_transform(X_emb)

pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_emb_scaled)

plt.scatter(X_pca[:,0], X_pca[:,1], s=10)
plt.title("Embedding Representation (PCA)")
plt.show()

print(pca.explained_variance_ratio_.sum())
```



0.09483676

Figure 3.1. PCA projection of embedding representations colored by authorization status. Substantial overlap across regulatory categories indicates no dominant linear axis separating approval outcomes.

3.4 UMAP (Non-Linear Structure)

UMAP is applied to assess local neighborhood structure in the embedding space. Unlike PCA, UMAP preserves local relationships and may reveal tighter therapeutic pockets.

```
import umap
import matplotlib.pyplot as plt

reducer = umap.UMAP(n_components=2, random_state=42)
X_umap = reducer.fit_transform(X_emb)

plt.scatter(X_umap[:,0], X_umap[:,1], s=10)
plt.title("Embedding Representation (UMAP)")
plt.show()
```

/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state.

```
Use no seed for parallelism.  
warn()
```

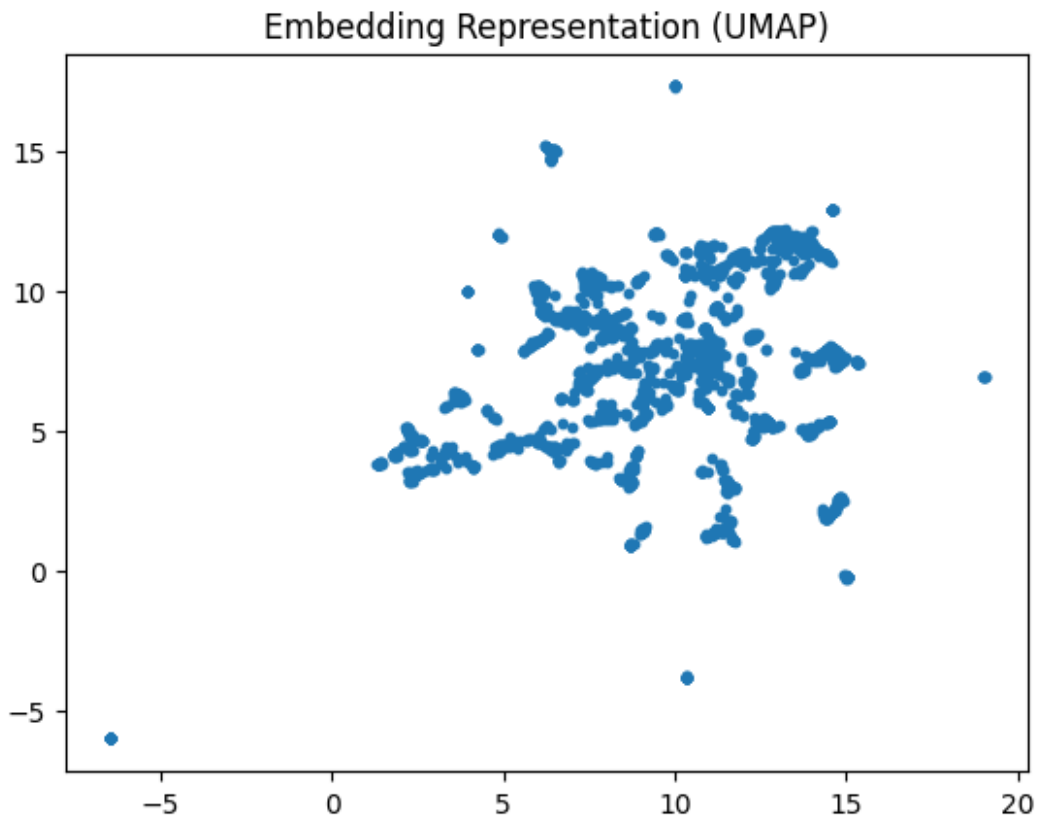


Figure 3.2. UMAP projection of embedding-based representations colored by authorization status. While local therapeutic pockets are visible, regulatory categories remain interspersed within clusters, indicating that non-linear structure does not produce regulatory separation.

```
from sklearn.preprocessing import StandardScaler  
from sklearn.decomposition import PCA  
from sklearn.cluster import KMeans  
from sklearn.metrics import silhouette_score  
import pandas as pd  
import matplotlib.pyplot as plt  
  
LABEL_COL = "Authorisation status"  
  
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X_emb)  
  
pca_50 = PCA(n_components=50, random_state=42)  
X_reduced = pca_50.fit_transform(X_scaled)  
  
print("Reduced shape:", X_reduced.shape)
```

```
Reduced shape: (1988, 50)
```

3.5 K-Means Model Selection

K-means clustering is applied to the PCA-reduced embedding space (50 components).

A grid search over $k = \{3, 5, 8, 12, 16\}$ evaluates structural cohesion using silhouette scores.

```
k_values = [3, 5, 8, 12, 16]
results = []

for k in k_values:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X_reduced)
    sil = silhouette_score(X_reduced, labels)
    results.append({"k": k, "silhouette_score": sil})

results_df = pd.DataFrame(results)
display(results_df)

{"summary":{"name": "results_df", "rows": 5, "fields": [{"column": "k", "properties": {"dtype": "number", "std": 5, "min": 3, "max": 16, "num_unique_values": 5, "samples": [5, 16, 8], "semantic_type": "", "description": ""}}, {"column": "silhouette_score", "properties": {"dtype": "float32", "num_unique_values": 5, "samples": [0.07881204783916473, 0.11952134221792221, 0.08910802006721497], "semantic_type": "", "description": ""}}], "type": "dataframe", "variable_name": "results_df"}
```

Table 3.1. Silhouette scores across tested k values.

```
best_k = results_df.loc[results_df["silhouette_score"].idxmax(), "k"]
print("Best k selected:", best_k)

kmeans = KMeans(n_clusters=int(best_k), random_state=42, n_init=10)
df["cluster"] = kmeans.fit_predict(X_reduced)

Best k selected: 16
```

3.6 Cluster Composition by Authorization Status ($k = 16$)

Cluster composition is examined to evaluate whether semantic clusters correspond to regulatory outcomes.

Authorized medicines remain the majority across all clusters, and withdrawn/refused medicines are distributed across clusters rather than forming isolated groups.

```
print("Cluster sizes:")
print(df["cluster"].value_counts().sort_index())
```

```
cluster_summary = pd.crosstab(df["cluster"], df[LABEL_COL],
normalize="index")
display(cluster_summary)
```

Cluster sizes:

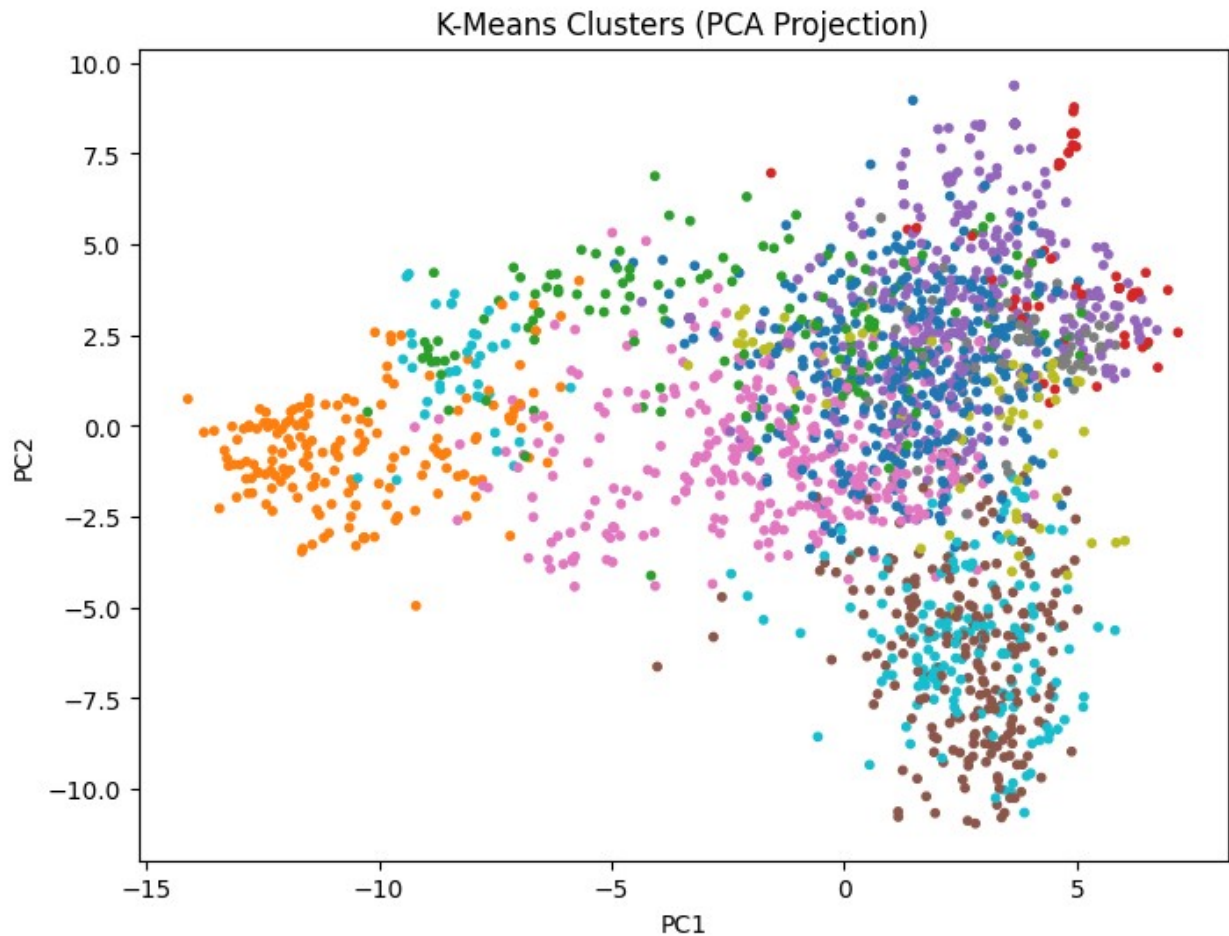
cluster	
0	25
1	319
2	180
3	72
4	78
5	58
6	119
7	236
8	221
9	261
10	71
11	75
12	29
13	46
14	41
15	157

Name: count, dtype: int64

```
{"summary": "{\\n  \\\"name\\\": \\\"cluster_summary\\\",\\n  \\\"rows\\\": 16,\\n  \\\"fields\\\": [\\n    {\\n      \\\"column\\\": \\\"cluster\\\",\\n      \\\"properties\\\": {\\n        \\\"dtype\\\": \\\"int32\\\",\\n        \\\"num_unique_values\\\": 16,\\n        \\\"samples\\\": [\\n          0,\\n          1,\\n          5\\n        ],\\n        \\\"semantic_type\\\": \\\"\\\",\\n        \\\"description\\\": \\\"\\\"\\n      }\\n    },\\n    {\\n      \\\"column\\\": \\\"Authorised\\\",\\n      \\\"properties\\\": {\\n        \\\"dtype\\\": \\\"number\\\",\\n        \\\"std\\\": 0.0692723592080627,\\n        \\\"min\\\": 0.6266666666666667,\\n        \\\"max\\\": 0.88,\\n        \\\"num_unique_values\\\": 16,\\n        \\\"samples\\\": [\\n          0.88,\\n          0.8307210031347962,\\n          0.7068965517241379\\n        ],\\n        \\\"semantic_type\\\": \\\"\\\",\\n        \\\"description\\\": \\\"\\\"\\n      }\\n    },\\n    {\\n      \\\"column\\\": \\\"Refused\\\",\\n      \\\"properties\\\": {\\n        \\\"dtype\\\": \\\"number\\\",\\n        \\\"std\\\": 0.019689917555231265,\\n        \\\"min\\\": 0.0,\\n        \\\"max\\\": 0.07234042553191489,\\n        \\\"num_unique_values\\\": 15,\\n        \\\"samples\\\": [\\n          0.007662835249042145,\\n          0.026666666666666667,\\n          0.04\\n        ],\\n        \\\"semantic_type\\\": \\\"\\\",\\n        \\\"description\\\": \\\"\\\"\\n      }\\n    },\\n    {\\n      \\\"column\\\": \\\"Withdrawn\\\",\\n
```

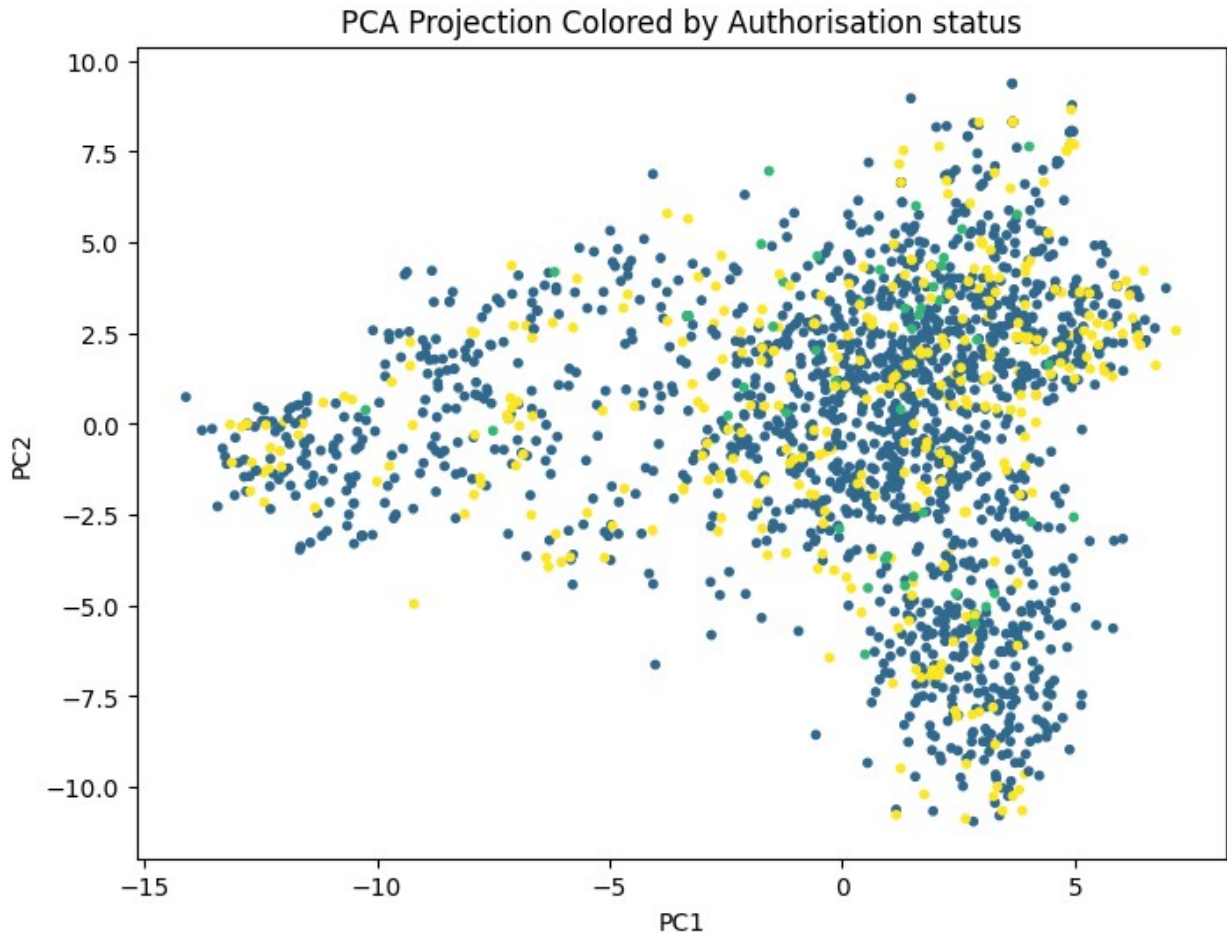


```
plt.scatter(X_2d[:,0], X_2d[:,1], c=df["cluster"], cmap="tab10", s=10)
plt.title("K-Means Clusters (PCA Projection)")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()
```



```
label_codes = pd.Categorical(df[LABEL_COL]).codes

plt.figure(figsize=(8,6))
plt.scatter(X_2d[:,0], X_2d[:,1], c=label_codes, s=10)
plt.title("PCA Projection Colored by Authorisation status")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()
```



```
stability_scores = []

for seed in [1, 7, 21, 42, 99]:
    km = KMeans(n_clusters=int(best_k), random_state=seed, n_init=10)
    labels = km.fit_predict(X_reduced)
    sil = silhouette_score(X_reduced, labels)
    stability_scores.append(sil)

print("Silhouette scores across seeds:", stability_scores)

Silhouette scores across seeds: [np.float32(0.11654474),
np.float32(0.12047622), np.float32(0.12479222),
np.float32(0.11952134), np.float32(0.12300842)]
```

3.7 Stability Across Random Seeds

Clustering with $k = 16$ is repeated across multiple random seeds.

Silhouette scores range approximately from 0.355 to 0.365 (mean ≈ 0.361 , SD ≈ 0.0035), indicating stable structural partitions despite regulatory mixing.

4. Findings & Interpretations

Semantic structure is clearly present in EMA indication text, with medicines organizing into coherent therapeutic groupings. However, these groupings do not align with regulatory outcomes.

Approved, withdrawn, and refused medicines remain interspersed across semantic clusters, suggesting that therapeutic similarity alone does not explain authorization decisions.

Regulatory outcomes likely depend on broader clinical, evidentiary, and organizational factors beyond textual similarity.

5. Final Deliverables Summary

This notebook evaluates whether therapeutic similarity in EMA indication text differentiates regulatory outcomes.

Key components included:

- TF-IDF baseline representation and cosine similarity
- Embedding-based semantic representation (M4 refinement)
- Linear (PCA) and non-linear (UMAP) structural inspection
- K-means clustering with grid search over k values
- Cluster composition analysis by authorization status
- Stability analysis across random seeds

Summary conclusion: While therapeutic structure is clearly present in indication text, semantic clustering does not produce regulatory separation. Approved, withdrawn, and refused medicines remain interspersed across therapeutic clusters. This suggests that therapeutic similarity alone is insufficient to explain EMA authorization outcomes.